4. Tools and System Prototype

The cancellation decision-support prototype connects booking-level risk estimation (Stage F), the capacity-aware decision policy (Stage G1) and an operational layer for staff: a web dashboard, an approval-guarded action store and an n8n workflow. It runs as a **retrospective demonstration**: 2017 City Hotel Lisbon bookings are replayed week by week as the model scored them. The dashboard, the action store and the n8n workflow are implemented and tested; no guest is contacted and no PMS, price or inventory record is changed. Section 4.5 separates what is built from what remains a production design.

4.1. Technology Stack

The stack separates predictive analysis, decision rules, data management and operational delivery. Python produces the scores, tiers and dated actions; CSV and JSON files carry them, with a run identifier, to the operational layer; PostgreSQL holds the relational data model and, optionally, the shared action store; Streamlit provides the staff interface; and n8n demonstrates how an approved action is handed to a workflow. Table 4.1 assesses each tool against the three criteria of the brief.

Table 4.1. Technology stack, role in the prototype and assessment

Tool	Role in the prototype	Scalability	Collaboration	Usability for managers
Python (pandas, scikit-learn, XGBoost)	Weekly scoring (F), G1 tiers and capacity-limited schedule, scenario analysis	Weekly batch of new bookings; reproducible runs	Run identifier and code / input hashes in each manifest	Invisible to managers; results reach them through the dashboard
CSV / JSON contract	Risk scores, action queue, dated schedule and G1 policy handed to the dashboard	Plain files now; same fields can move to tables or an API	Human-readable, versioned with the code	Can be opened in a spreadsheet for review

Tool	Role in the prototype	Scalability	Collaboration	Usability for managers
PostgreSQL 16	Hotel data model (10 tables, 2 views); shared action store with the same rules as SQLite	Constraints, indexes, multi-user access; writes serialised per store	One shared source for Reservations, Revenue Management and Finance	Managers never write SQL
SQLite	Local action store for the demo (default when no PostgreSQL address is set)	Single machine only	Not shared; one file per model run and policy	No set-up needed for a first demo
Streamlit	Staff dashboard: weekly queue, booking explorer, schedule, cancellation buffer, approvals, KPIs	Fine for a pilot; needs login and a server deployment at scale	Python code in the same repository	Filters, tables and buttons; no coding
n8n	Demo workflow: webhook, approval and execution guard, mock dispatch, reply	Workers and queues can be added in production	Visual nodes that operations staff can review	Readable sequence of steps
unittest + Node.js	Automated tests of data contract, guard rules, concurrency, workflow code and app	Runs in about a minute	Same checks for every team member	Not used by managers

4.2. Data Integration and Analytical Outputs

The selected XGBoost model (f-v3.1-2026-10-19) scores the bookings created in each booking week and still active at the end of that week. For 2017 this produced **13,156 risk-score records: 12,102 individual bookings scored by XGBoost and 1,054 group-block bookings scored by the group rule** (segment × deposit), which go to the capacity track. Each record holds the booking identifier, scoring time, model version, cancellation probability, risk tier, expected revenue at risk and days to arrival. G1 then re-tiers the 12,102 individual bookings and turns them into a dated, capacity-limited action schedule (Table 4.2).

Table 4.2. Analytical outputs used by the prototype (2017 evaluation period)

Output	Recorded result	Operational use
Risk scores	13,156 records (12,102 individual, XGBoost; 1,054 group rule)	Estimated cancellation probability and revenue exposure per booking
Decision queue (G1)	12,102 bookings: I1 = 5,116; I2 = 5,186; I3 = 1,761; I5 = 39; 17 call tasks without a free slot go to manual review	Rule, tier and owner per booking
Action schedule	20,563 dated actions: 5,116 confirmation e-mails, 13,703 reminders, 1,744 call tasks	What is due on each day, imported into the action store
Decision policy	Calls: top 15% of each weekly batch by risk × booking value; contact: 60% of the batch; at most 20 calls a day, within 7 days of the decision and at least 8 days before arrival; G3 overbooking disabled	Keeps contact within staff capacity
Database structure	Hotel data model: 10 tables and 2 views, loaded and checked by SQL scripts (30 of 30 checks pass). Action store: 6 tables with guard triggers, on SQLite or PostgreSQL	Bookings, scores, actions, approvals, interventions and guest responses

Before showing anything, the dashboard checks that the model and policy files come from the same run, re-derives every G1 tier from the model scores, checks each dated action against the policy and refuses any field that is only known after the outcome. The outcome-based evaluation file is never read. Monetary figures shown to staff (expected revenue at risk; net value at risk on simulated costs) are scenario estimates, not realised or recovered revenue.

4.3. System Architecture and Workflow

The architecture follows the flow from booking data to risk estimation, decision policy and staff action (Figure 4.1). The analytical stages run in Python as weekly batches and hand their results over through the file contract. In the operational layer, staff work in the dashboard; every proposal, approval, execution, guest response and closure is written to

the action store, whose rules are enforced both in the service and by database triggers. n8n receives an action only after those checks have passed.

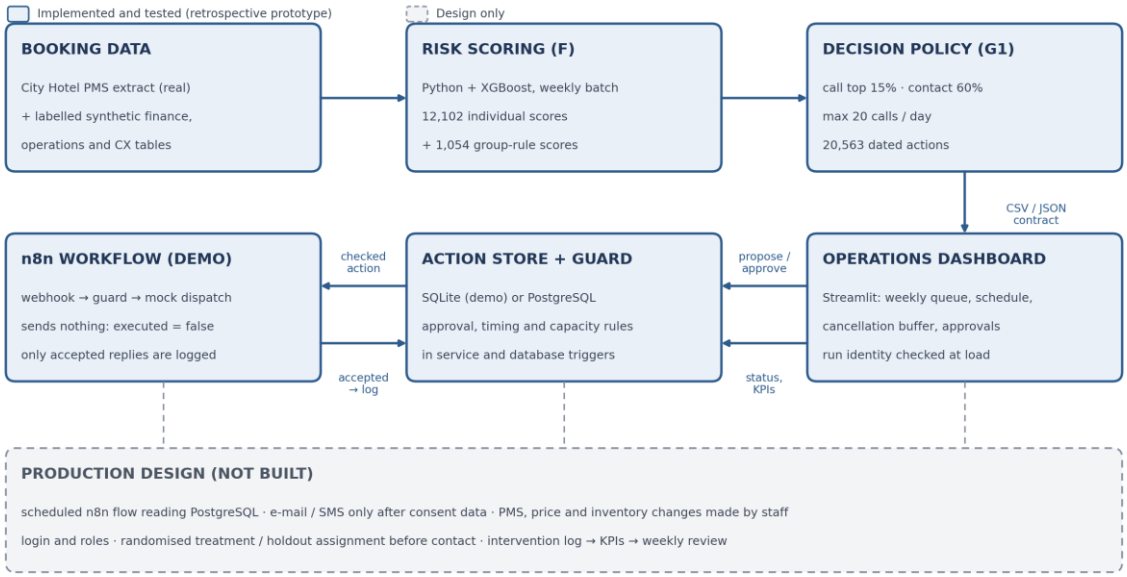


Figure 4.1. Technical architecture and booking-level decision workflow (solid: implemented; dashed: production design).

Before any action is executed, the store checks that it is approved (or automatic), not yet executed or closed, due on the replay date and before arrival; that a call is still inside the G1 window and the daily limit of 20; that the guest has not already responded; and that no automated message goes out on the same day as a call. Consent, other open actions and whether the booking is still active are not in the data, so staff must confirm them for each guest action. These checks, the call to n8n and the log entry form one database transaction, so two staff members working at the same time cannot exceed the call limit or both close and execute the same action. The logs are append-only.

4.4. Dashboard and Automation Process

The dashboard is organised around the work queue rather than model diagnostics. Seven tabs cover the weekly queue (Figure 4.2), a booking explorer filtered by track, risk tier, rule and market segment, the action schedule by due date, the cancellation buffer per arrival date × room type, actions and approval (filtered by approval status), KPIs such as approval

turnaround, and an About page. A replay date in the sidebar stands for "today"; nothing can be executed before its due date.

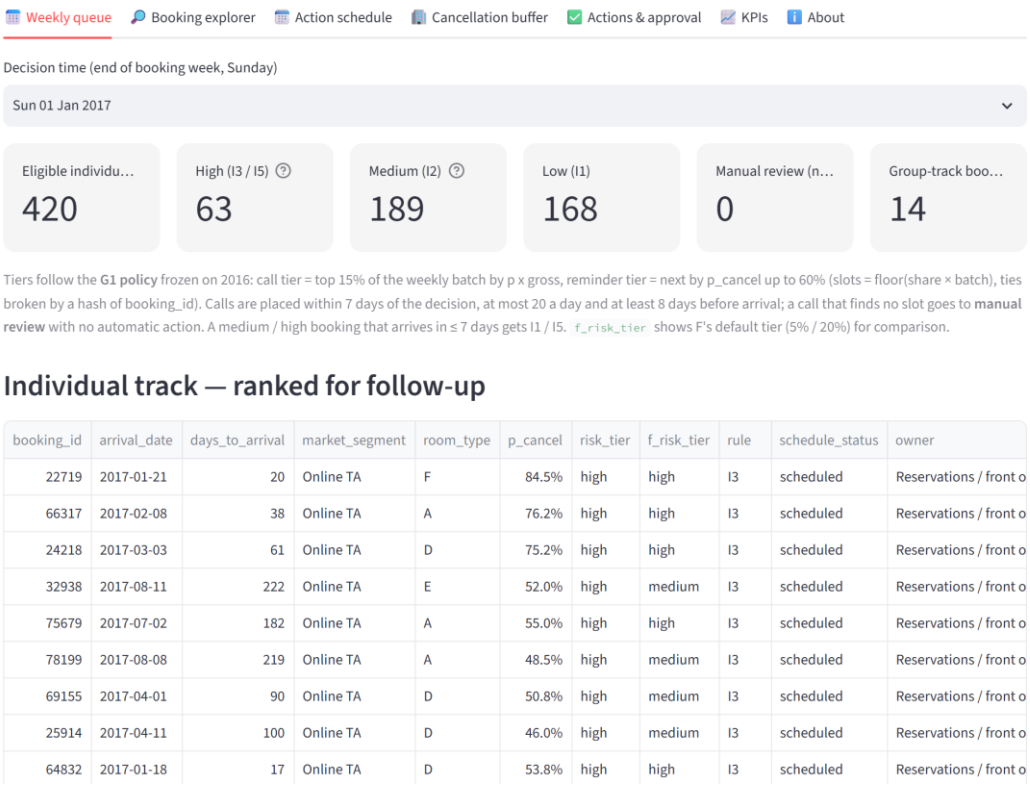


Figure 4.2. Weekly queue for the batch of 1 January 2017: 63 call-tier, 189 reminder-tier and 168 low-tier bookings.

Table 4.3. Intervention routing and control rules

Rule	Operational handling	Control measure
I1 – Standard	Confirmation on the decision day and a reminder 7 days before arrival (current practice).	Automatic; stopped once the guest responds.
I2 – Reminder	Personalised e-mail / SMS at 30 and 14 days before arrival (one at once if a milestone has passed).	Maximum 2; never on the same day as a call; stopped by any guest response.
I3 – Call	Call task for Reservations within 7 days of the decision, highest risk × value first.	Maximum 20 calls a day and 2 attempts per booking; outside the window: manual review or close without contact.

Rule	Operational handling	Control measure
I4 – Offer change	Date or room change after a call in which the guest asked to change.	Revenue Manager approval; tied to that exact request, a newer request needs a new offer.
I5 – Buffer only	Call-tier booking with 7 days or fewer to arrival: no contact, counted in the cancellation buffer.	Revenue Manager review of the buffer.
I6 – Deposit	Deposit request for a high-tier, no-deposit booking.	Off by default; needs a written policy and Revenue Manager approval.
G1 / G2 – Group track	Capacity allocation, resale or price adjustment per arrival date × room type.	Always pending until a named Revenue Manager approves.
G3 – Overbooking	Not available.	Disabled by policy; refused by the service, n8n and the database.

Approval is enforced in the database as well as in the interface: a sensitive action always starts as "pending", cannot be executed until a named reviewer approves it (Figure 4.3), and an approval decision cannot be reversed or changed afterwards.

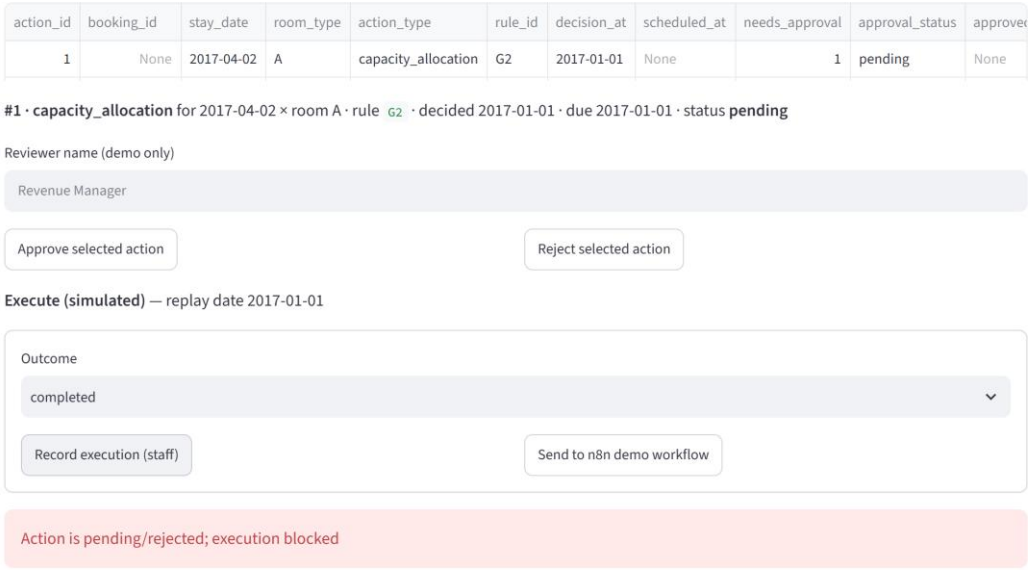


Figure 4.3. A pending capacity-allocation action is blocked until the Revenue Manager approves it.

For automatic and approved actions, staff can either log the execution themselves or hand it to the n8n workflow. n8n re-checks the approval and timing rules, prepares what it would send (an e-mail preview, a staff task or an approval notice) and replies; only a reply with "accepted: true" is logged, as executed by n8n (Figure 4.4). In this prototype the workflow sends nothing ("executed": false).



Figure 4.4. A call task handed to the local n8n workflow: accepted, logged by n8n, nothing sent.

4.5. Verification and Limitations

Table 4.4. Verification of the prototype

Check	Result
Automated tests, SQLite store	211 tests: all pass (5 skipped: 2 need a live n8n, 3 need PostgreSQL)
Action-store tests on PostgreSQL 16	172 tests: all pass (4 skipped: 2 SQLite-file only, 2 need a live n8n)
Live n8n workflow (local)	Approved call accepted and logged; pending action and overbooking refused by n8n's own guard
Concurrent staff sessions	Last call slot goes to one session only; an action cannot be both closed and executed; one action is sent to n8n once
Database scripts	Data load 30 of 30 checks; approval-guard migration 19 of 19 checks

Limitations. The prototype has no login or user roles, holds no guest contact details and contacts no one. Consent and open actions are unknown in the data and must be confirmed by staff. The n8n workflow is a demonstration: a production version would run on a schedule, read due actions from PostgreSQL, send e-mail or SMS only once consent data exist, and leave PMS, price and inventory changes to staff. Treatment and holdout assignment for the pilot, which is needed to measure the real effect of contacts, is designed but not implemented. All financial figures remain scenario estimates.